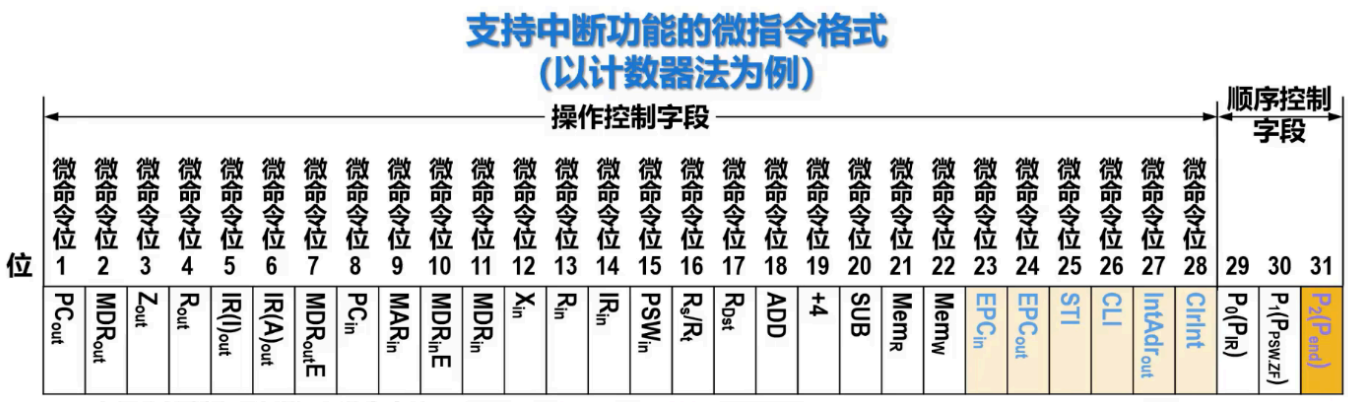
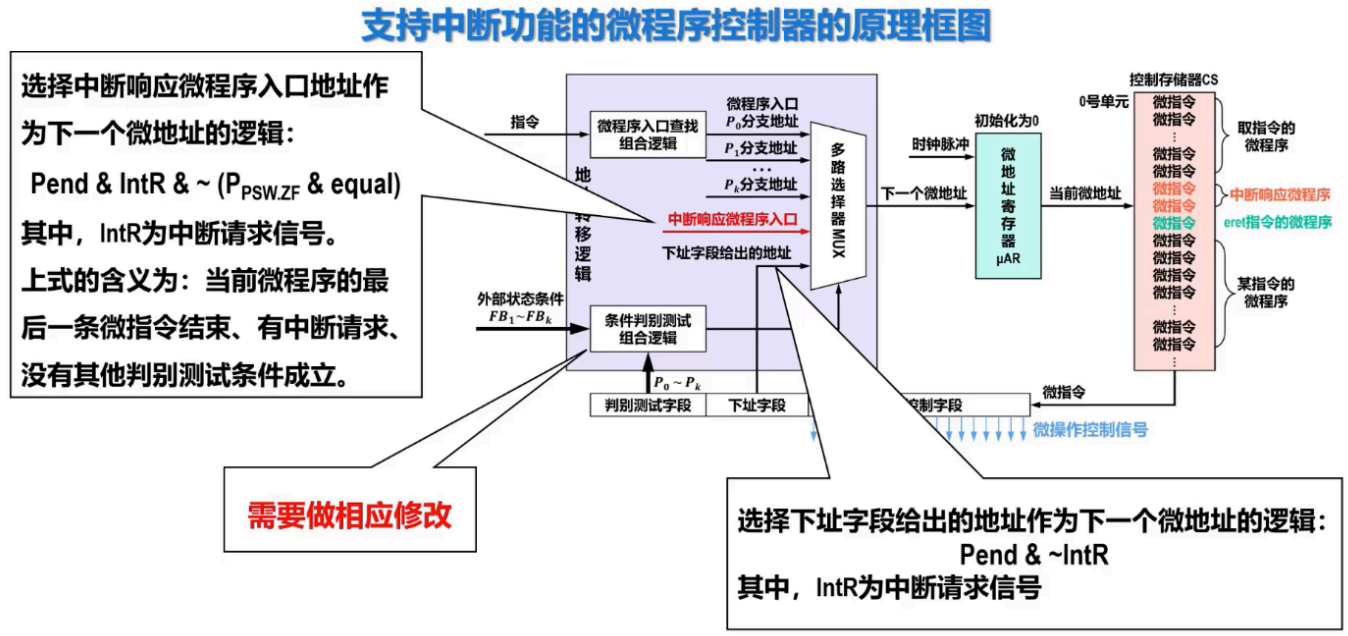
一、字段增加



- 向操作控制字段新增6个微命令位：EPC_{in}，EPC_{out}，STI，CLI，$IntAdr_{out}$，ClrInt
- 由于指令执行完毕后要进行中断请求的检查，因此不管采用下址字段法还是计数器法，都必须在顺序控制字段中新增一个微指令结束判别测试位$P_2(P_{end})$，用于指明当前微指令为最后一条微指令，并根据中断测试条件进行分支。

- 基于上述支持中断功能的微指令格式，重新设计微指令并新增3条微指令（2条用于中断响应操作、1条用于中断返回）。
- 有了微指令，就可以为指令集中的每条指令重新编写相应的微程序以实现指令功能。
- 相应地，还需要修改微程序控制器的地址转移逻辑。

改造后的原理框图



这张图的核心变化是：在原有"下址字段+MUX"的微程序控制器框架上，**为MUX新增了一路输入——中断响应微程序入口**，并在每条机器指令的**执行微程序结束时增加一次"是否响应中断"的判断**

1. 控制存储器CS的新布局

右侧的控存被划分为几段功能区域，与传统微程序控制器相比**新增了两段**：

- **0号单元起**：取指令的微程序（所有机器指令共享的取指周期，μAR上电复位强制指向这里）
- **中断响应微程序**（红色标注，**新增**）：对应数据通路图中"中断隐指令"要完成的关中断、保存断点、送中断入口到PC这一整套操作
- **eret指令的微程序**（绿色标注，**新增**）：对应中断返回指令的微程序，负责恢复断点EPC→PC

- **其他段**：各机器指令各自的执行周期微程序
这两段新增微程序的存在，使中断的"响应"与"返回"都纳入了微程序的统一解释框架

2. 微程序入口查找组合逻辑：新增一路输出

这个组合逻辑原本的功能是：根据IR中的操作码，给出**当前指令执行周期微程序**的入口地址（对应图中P_1、P_1'...P_k分支地址）
现在的修改是：**新增输出一路"中断响应微程序入口"**，作为一个独立的常量地址接到MUX的一个输入端，与各指令的微程序入口并列供MUX选择——左上角红字"需要做相应修改"指的就是这里要扩展功能

3. 两个关键逻辑表达式

左上角标注：选择中断响应微程序入口的条件

```
Pend & IntR & ~(P_PSW.ZF & equal)
```

三个条件同时成立时，MUX才选择"中断响应微程序入口"作为下一个微地址：
- **Pend**：当前微程序**最后一条微指令**的标志信号（新增的判别位），表示机器指令的执行微程序已经走完
- **IntR**：中断请求信号，来自数据通路图中中断控制逻辑发出的请求
- **~(P_PSW.ZF & equal)**：当前微指令本身没有其他判别测试条件成立（例如beq指令需要根据ZF做分支测试），避免与正常分支冲突
这三个条件恰好对应"中断响应的三要素"：**指令边界已到、有请求未被屏蔽、当前不处于其他条件分支**——与408教材讲的"CPU响应中断的三个必要条件"一一对应

右下角标注：选择下址字段的条件

```
Pend & ~IntR
```

当前微程序执行到最后一条微指令且**没有**中断请求时，按传统逻辑选择下址字段给出的地址（通常就是回到取指微程序的0号单元），开始下一条指令的取指
两个表达式**互斥**设计，保证MUX在同一时刻只选一路输入，避免微地址冲突

4. MUX的多路选择器角色

MUX是这张图的**决策中枢**，它汇集了几路可能的"下一个微地址"来源：
1. 来自**下址字段**给出的地址（正常顺序执行）
2. 来自**条件判别测试**修改后的地址（P_0~P_k根据PSW等状态做的分支跳转）
3. 来自**微程序入口查找组合逻辑**给出的各指令微程序入口（操作码译码）
4. **新增**：来自同一查找逻辑的**中断响应微程序入口**
MUX的选择信号由左侧的两个逻辑表达式共同驱动，通过条件组合决定下一拍把哪个地址装入μAR

5.完整流程串讲

```
上电复位 → μAR强制清0
    ↓
从CS的0号单元开始执行取指微程序
    ↓
指令装入IR，操作码送入微程序入口查找组合逻辑
    ↓
MUX根据操作码选择该指令执行微程序的入口 → μAR
    ↓
逐条执行该指令的执行微程序
    ↓
最后一条微指令（Pend=1）→ MUX做关键判断：
    ├─ IntR=1 且无其他测试条件 → 选"中断响应微程序入口"
    └─ IntR=0                  → 选"下址字段"（回0号单元）
    ↓
若进入中断响应微程序：
    逐条微指令发出控制信号：关中断(IE←0)、PC→EPC、
    MUX选中断入口→PC、清中断请求(ClrInt)
    （这些信号正是数据通路图上那四条蓝线）
    ↓
CPU开始执行ISR的机器指令（跳出微程序层面）
    ↓
ISR末尾执行eret指令
    ↓
eret的微程序执行：EPC→PC（恢复断点）
    ↓
回到取指微程序（μAR=0），循环往复
```